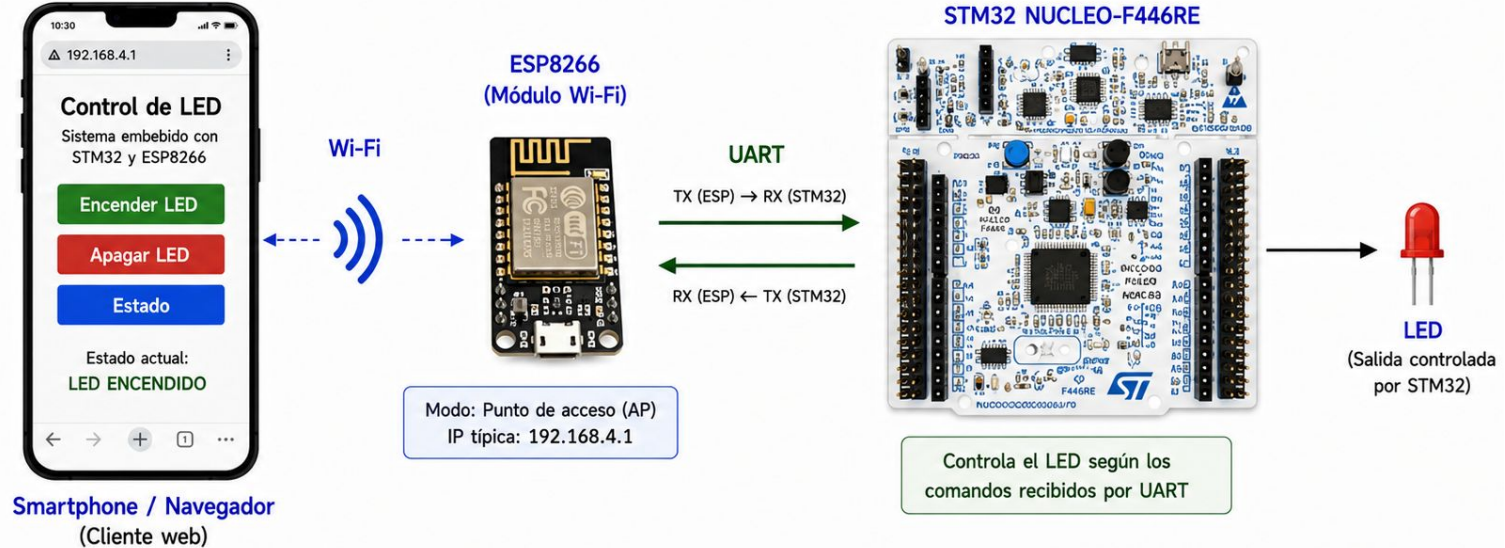


Periféricos y Comunicaciones en Sistemas Embebidos

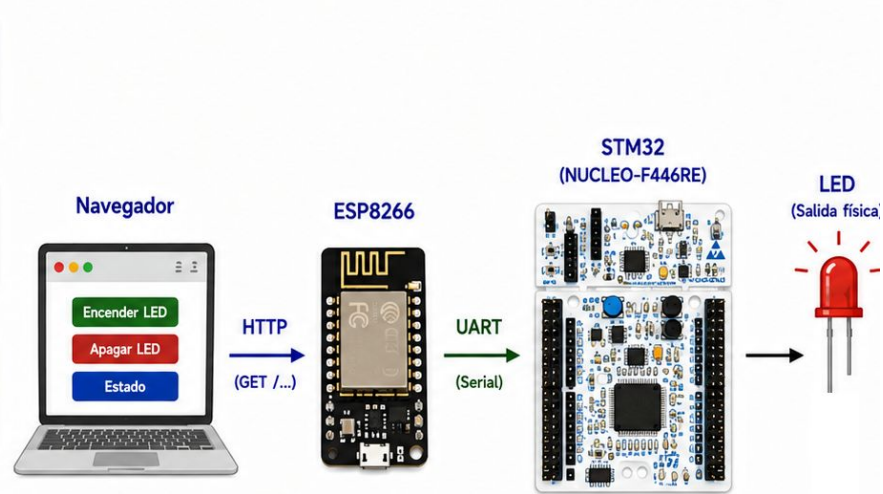
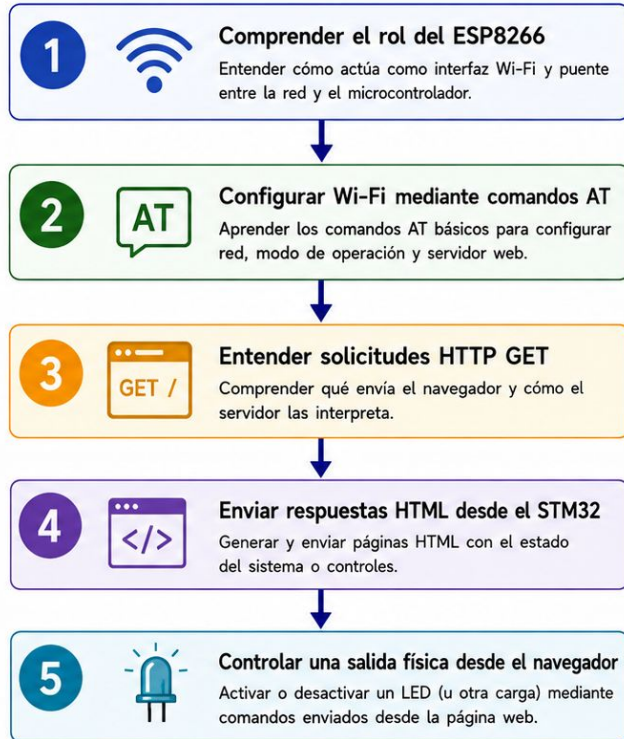
clase 11: 04/08/2026

Servidor web embebido con STM32 y ESP8266



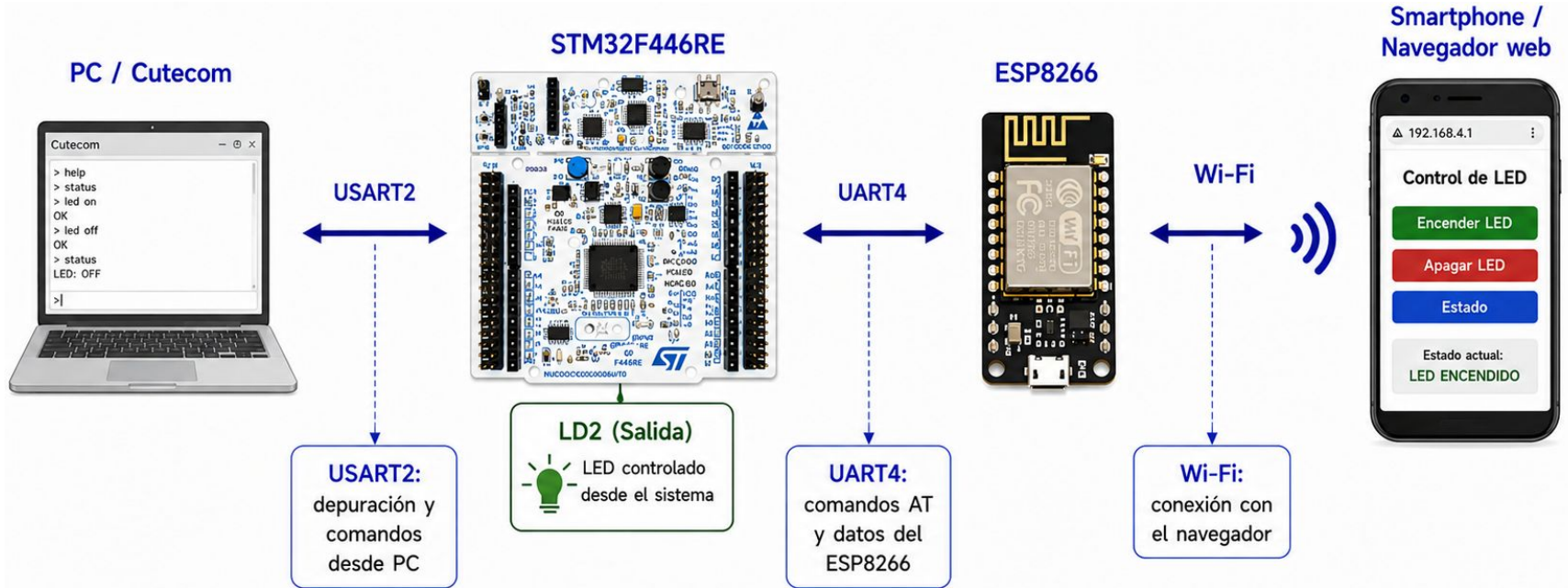
Una **página web** puede convertirse en una **interfaz de control** para un **sistema embebido**.

Objetivos de la clase



Primero **entendemos** la **comunicación**;
luego analizamos el **código**.

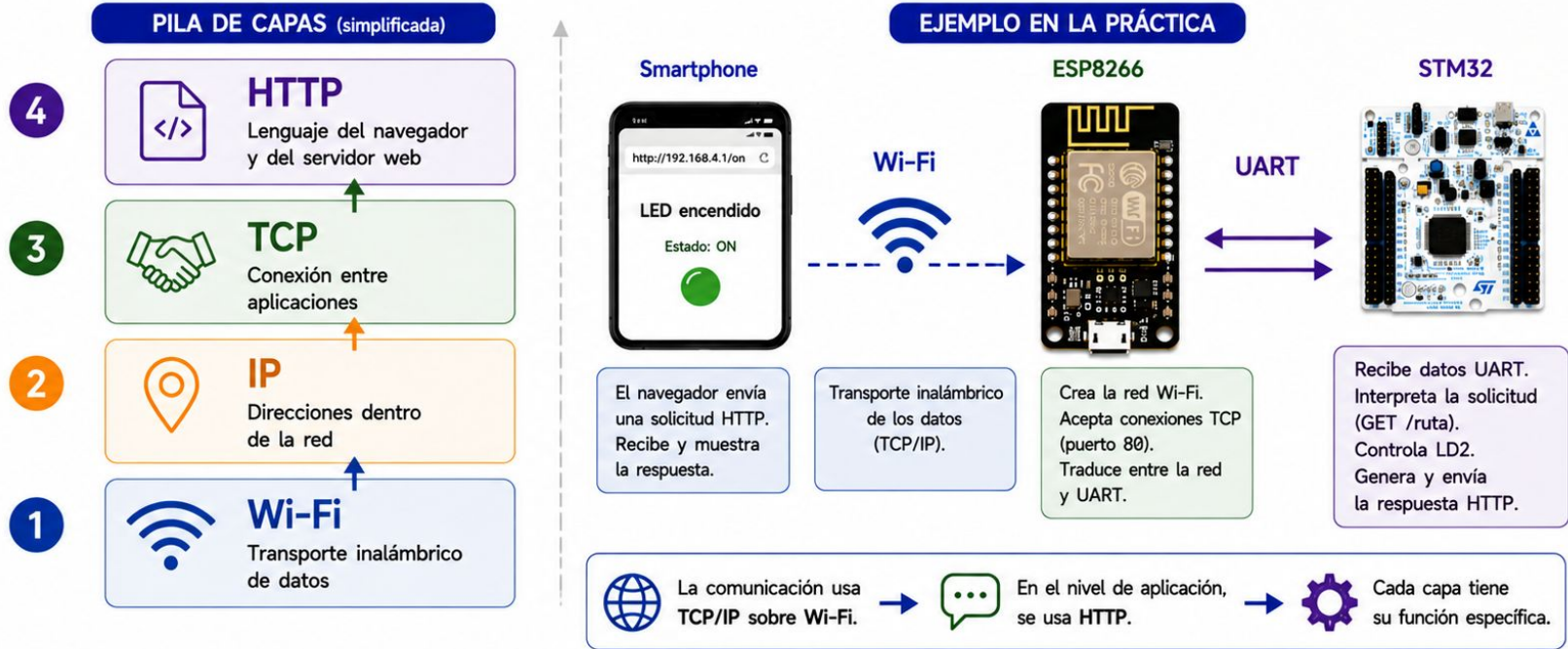
Arquitectura general del sistema



El **STM32** no maneja **Wi-Fi** directamente; dialoga con el **ESP8266** por **UART**.

No todo es lo mismo: Wi-Fi, TCP/IP y HTTP

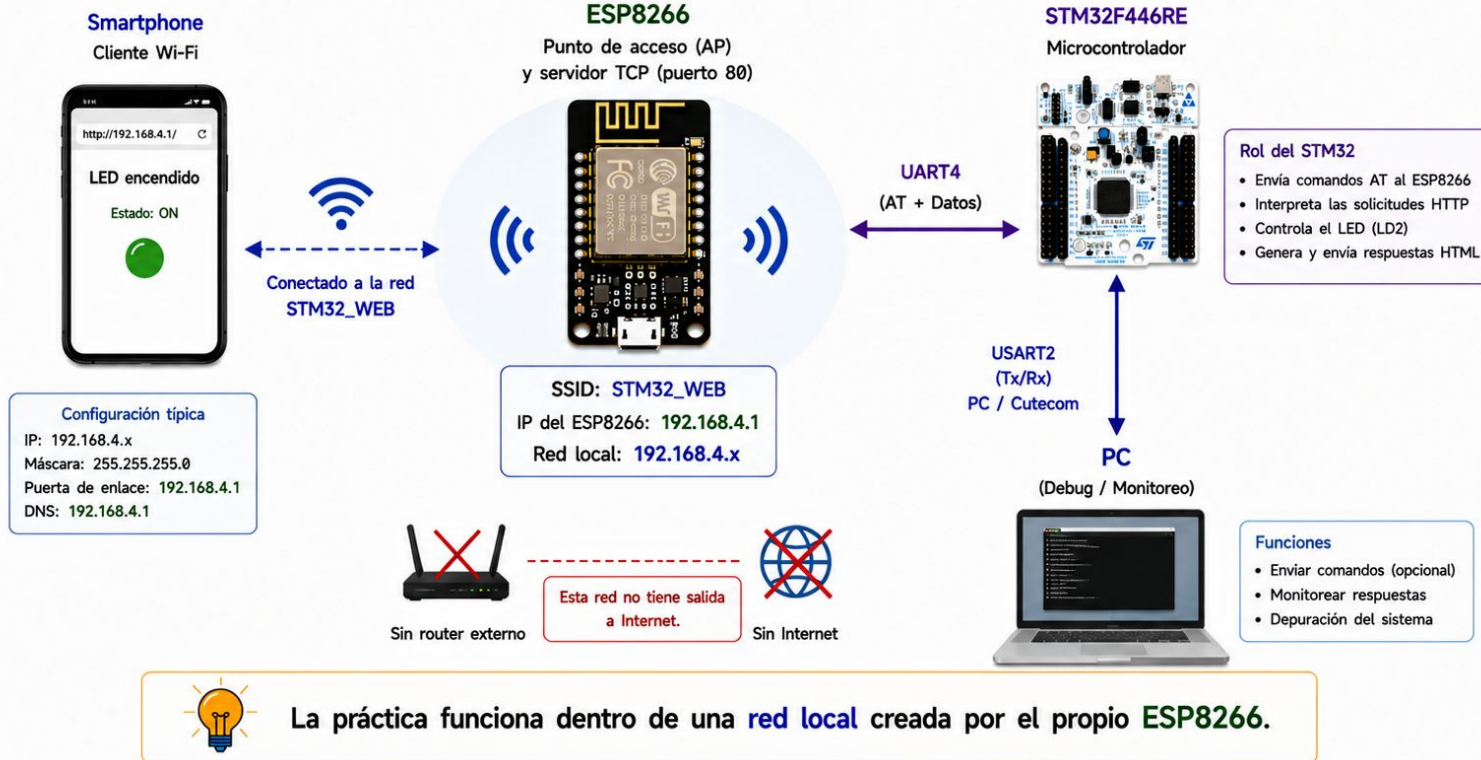
Diferentes capas, diferentes funciones. Todas trabajan juntas.



Wi-Fi conecta dispositivos; **HTTP** define qué se dicen.

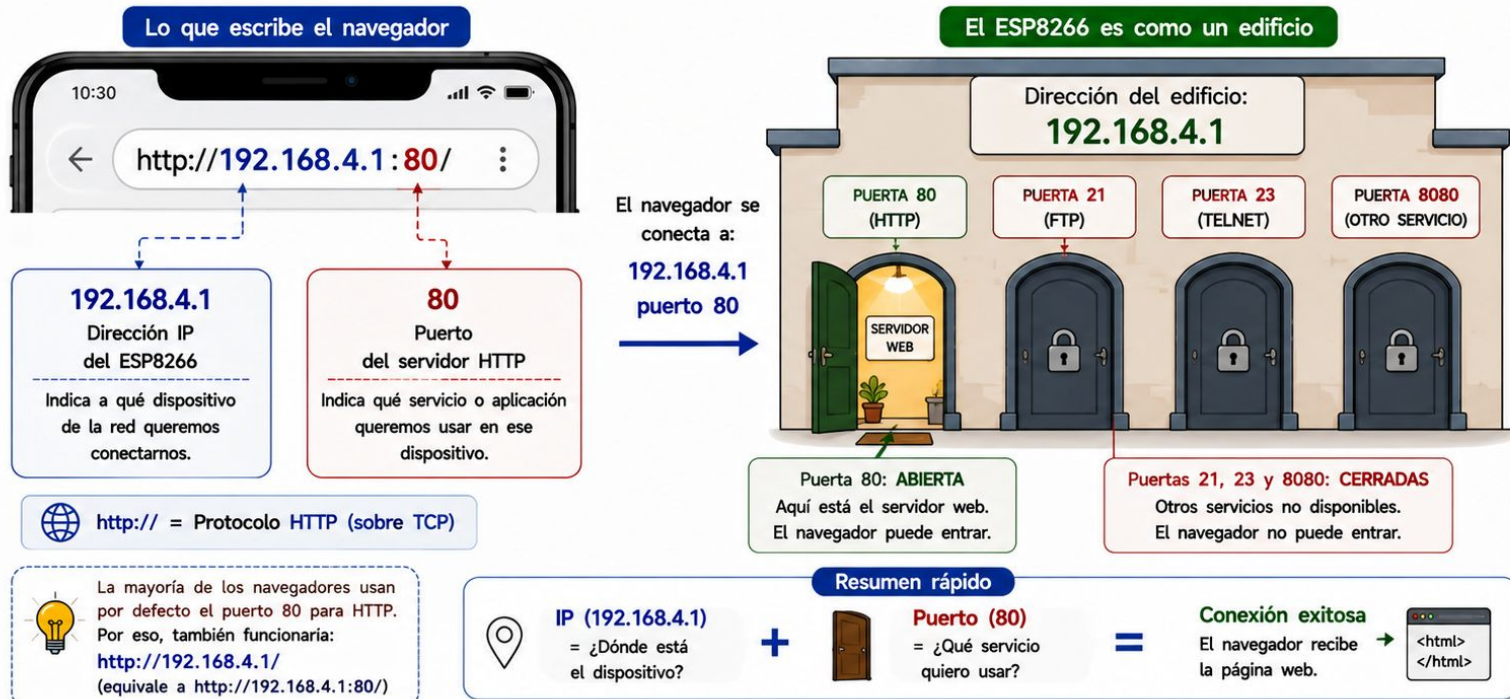
Una red local creada por el ESP8266

El ESP8266 crea su propia red Wi-Fi y actúa como servidor web.



IP y puerto: ¿a dónde se conecta el navegador?

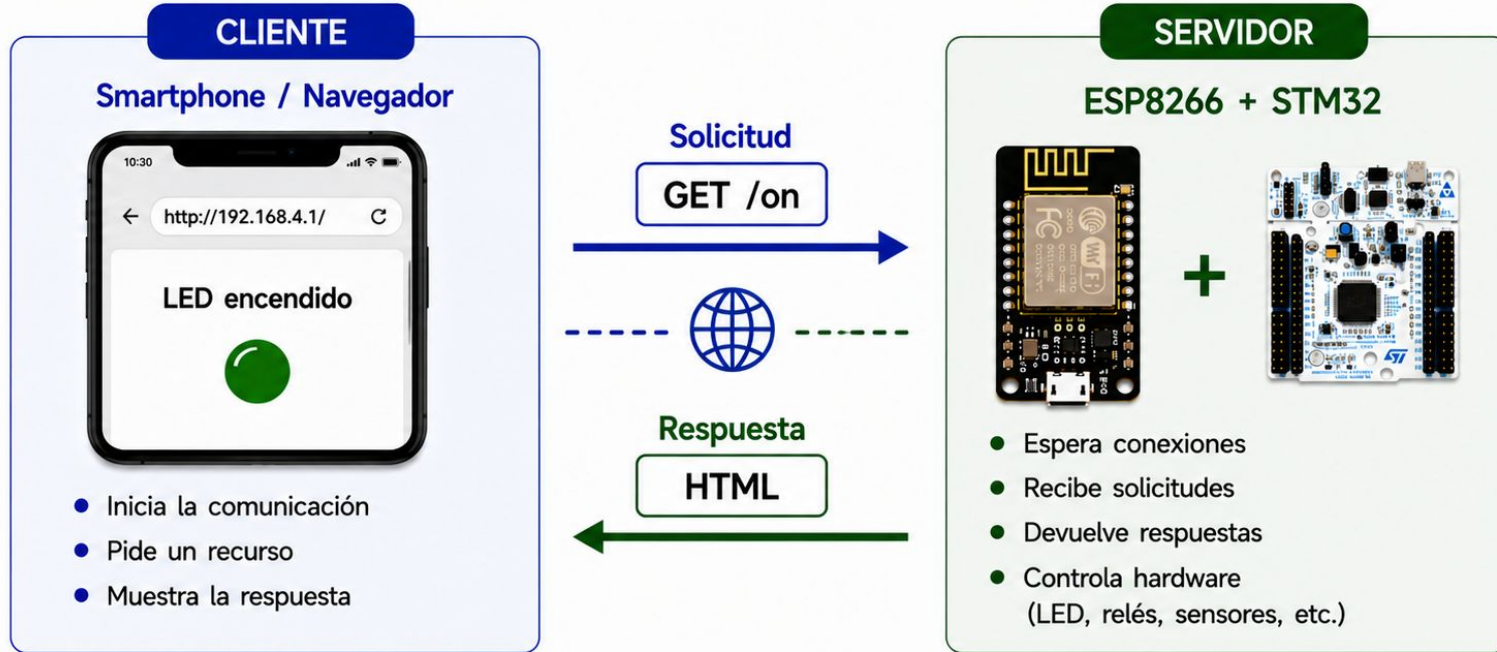
El navegador se conecta a la IP del ESP8266 en un puerto específico.



La **IP** identifica el dispositivo; el **puerto** identifica el **servicio**.

Modelo cliente-servidor

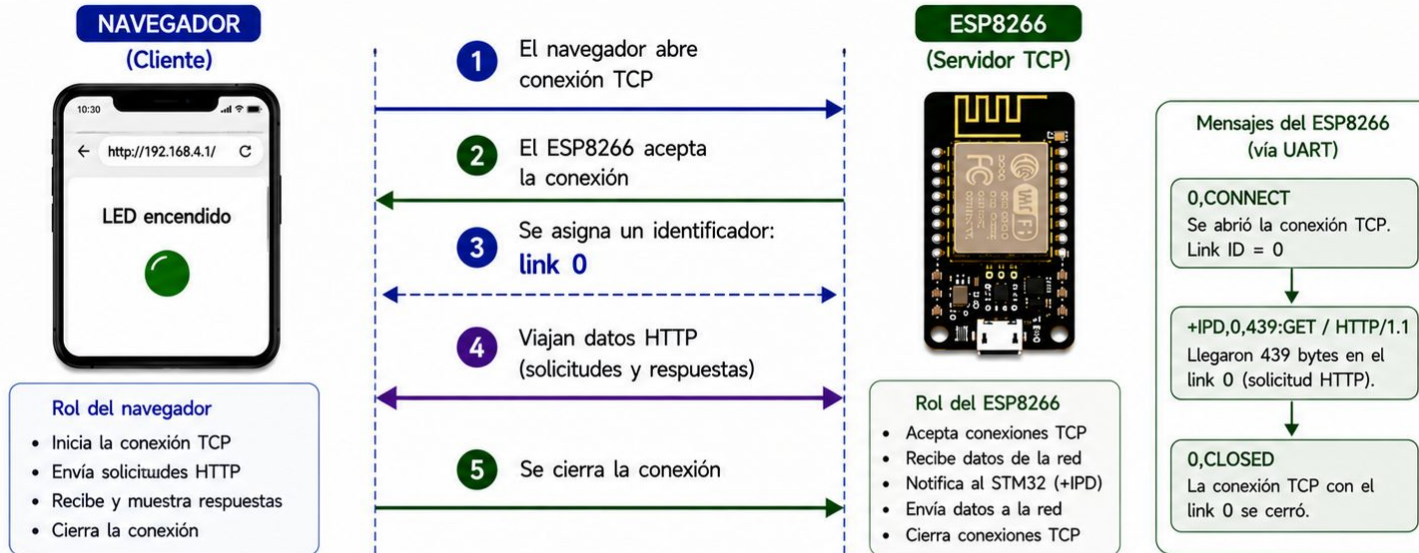
La comunicación se basa en solicitudes y respuestas.



El **cliente** pide; el **servidor** responde.

TCP: una conexión entre aplicaciones

TCP establece una conexión lógica entre el navegador y el ESP8266 para intercambiar datos.



Aplicación cliente
(Navegador)

Conexión TCP
(link 0)



Servidor TCP
(ESP8266)

Características de TCP

- Conexión orientada
- Confiable y ordenada
- Un canal por conexión (link ID)



TCP crea un canal de comunicación antes de transportar HTTP.

HTTP: el lenguaje del navegador

HTTP define cómo el navegador pide recursos y cómo el servidor responde.

Ejemplo de solicitud HTTP (simplificada)

GET /on HTTP/1.1

Host: 192.168.4.1

Connection: keep-alive

\r\n

i Las líneas terminan con CRLF (\r\n).
La línea vacía final indica el fin de los encabezados.

Métodos HTTP comunes

Método	Significado
GET	Pide un recurso (leer)
POST	Envía datos para crear o actualizar
PUT	Actualiza un recurso
DELETE	Elimina un recurso

Ejemplos de rutas

- / → página principal
- /on → encender LED
- /off → apagar LED
- /state → estado actual

Método

Qué acción desea realizar.
Ejemplo: GET

Ruta

Recurso solicitado en el servidor.
Ejemplo: /on

Versión

Versión del protocolo HTTP.
Ejemplo: HTTP/1.1

Destino solicitado

Indica a qué servidor se hace la solicitud.

Headers (encabezados)

Información adicional sobre la petición.
Ejemplo: Connection

En el STM32

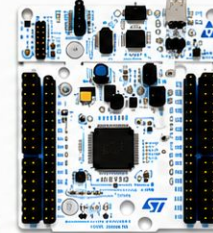
1 El STM32 recibe la solicitud HTTP desde el ESP8266.

2 Busca la ruta en la solicitud.
Ejemplo buscado: **GET /on**

3 Si coincide con /on, ejecuta la acción:

 **Enciende LD2**
(LED verde en la placa)

4 Genera una respuesta HTTP (HTML) y la envía al navegador.



El navegador muestra la respuesta

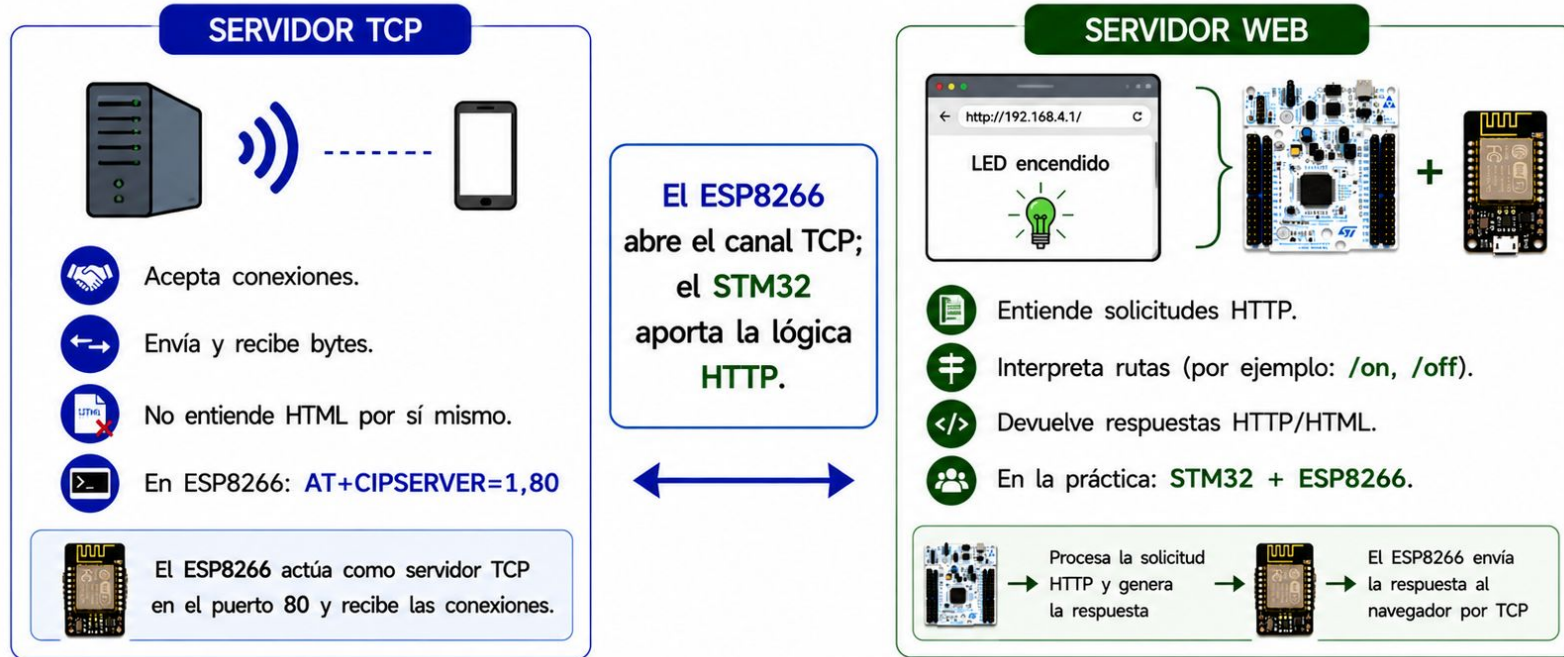
El HTML recibido se interpreta y se muestra en la pantalla del navegador.



HTTP es **texto estructurado** que el **firmware** puede interpretar.

Servidor TCP vs servidor web

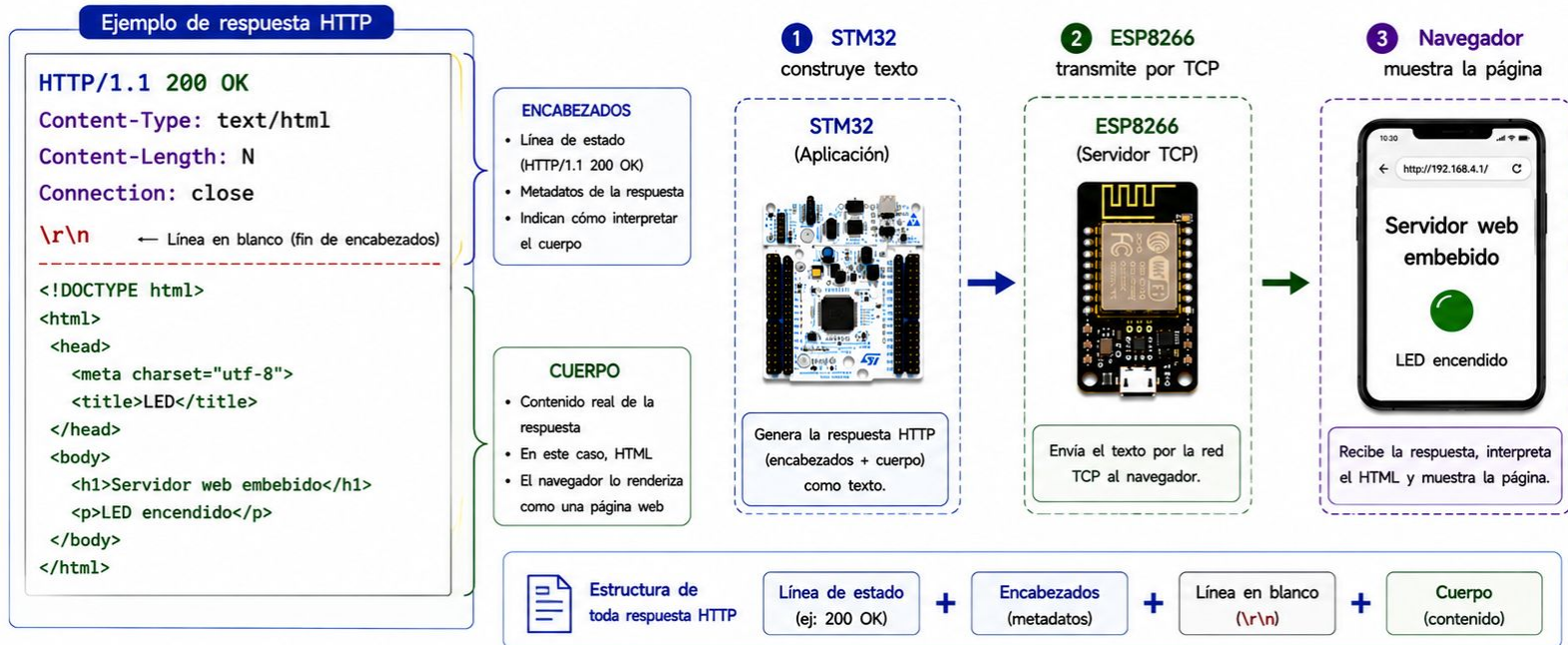
Dos conceptos diferentes que trabajan juntos.



Uno provee el **canal**, el otro entiende el **lenguaje** y **controla el sistema**.

Respuesta HTTP: encabezados y cuerpo

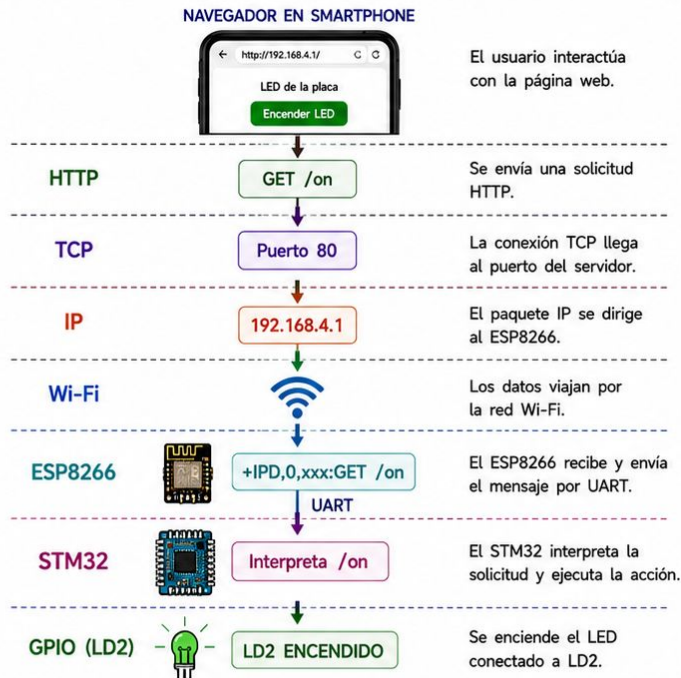
El servidor web responde con texto estructurado: primero los encabezados, luego el cuerpo (HTML).



El navegador necesita una respuesta HTTP válida para mostrar la página.

De la página web al LED

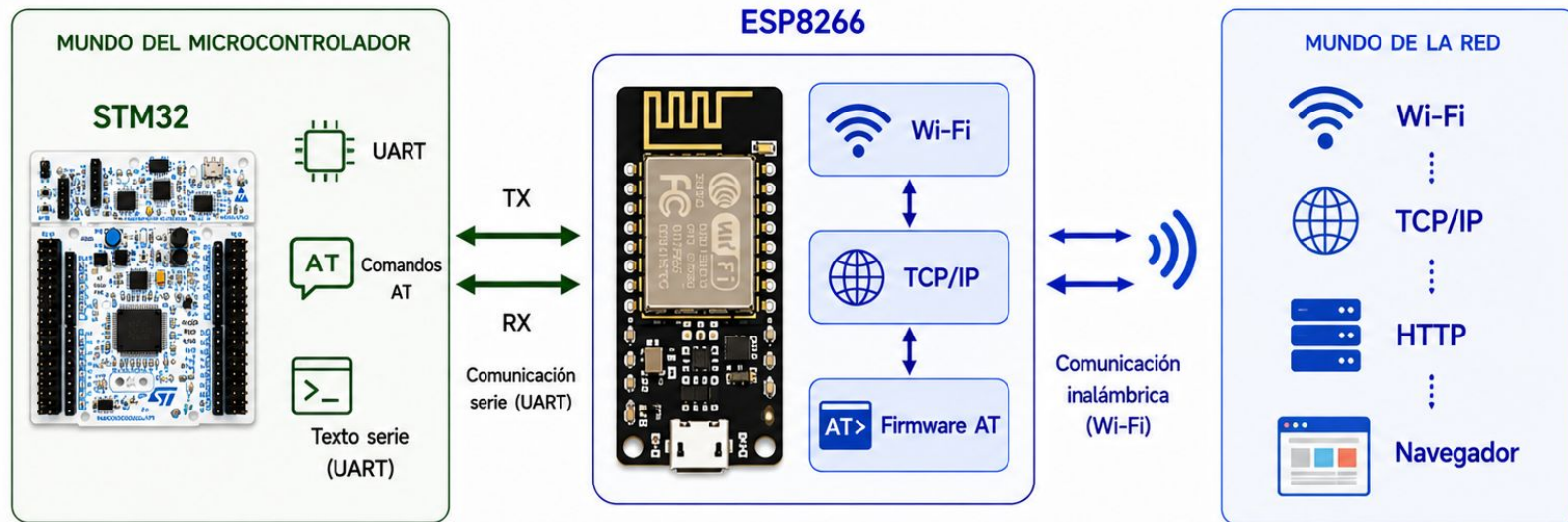
Una acción en el navegador atraviesa varias capas hasta controlar el LED.



Cada **capa** resuelve una **parte** del problema.

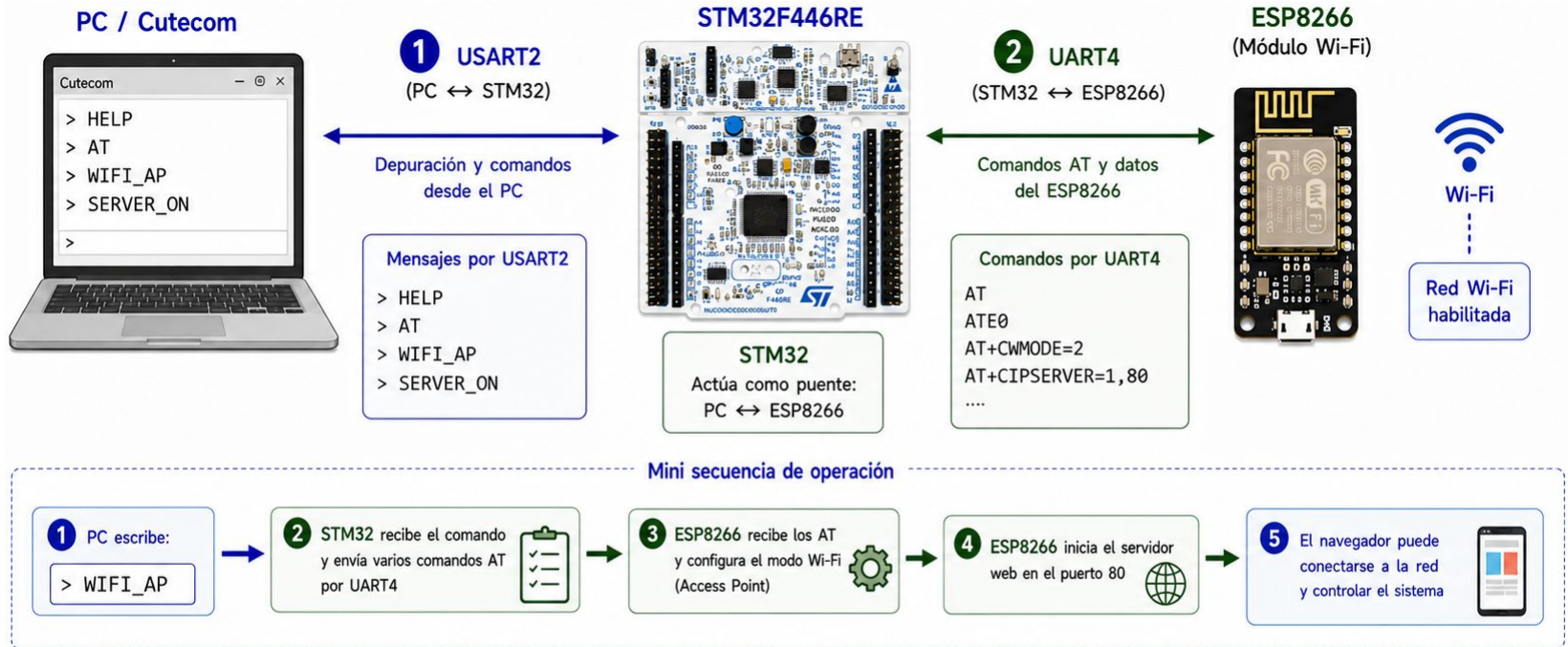
¿Qué rol cumple el ESP8266?

El ESP8266 actúa como un puente entre dos mundos



El **ESP8266** transforma comandos serie en acciones de red.

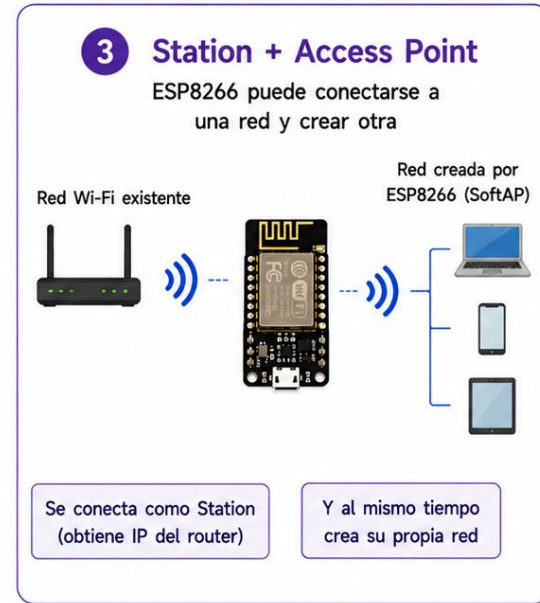
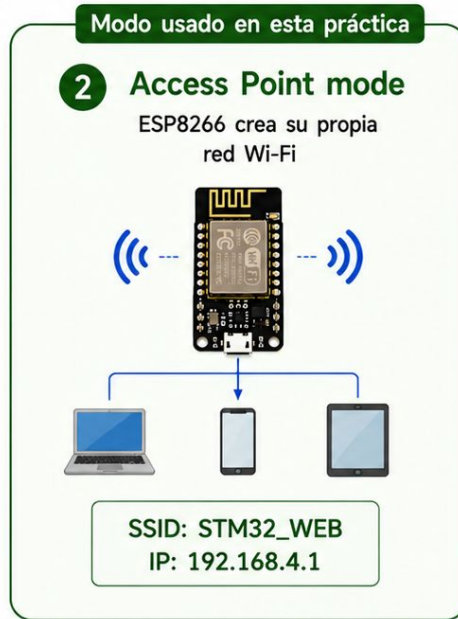
UART como canal de control



La **UART** transporta **texto**; el **significado** lo define el **firmware**.

Modos Wi-Fi del ESP8266

El ESP8266 puede trabajar en distintos modos de operación.



Usamos **SoftAP** para que la práctica no dependa de un **router externo**.

Dirección IP del servidor embebido

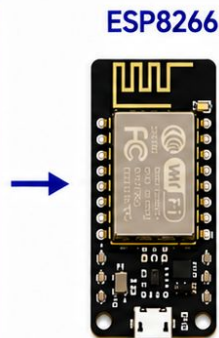
El ESP8266 crea una red Wi-Fi y actúa como servidor web.

**SSID:**
STM32_WEB

**Password:**
12345678

**IP del ESP8266:**
192.168.4.1

**Navegador:**
http://192.168.4.1/

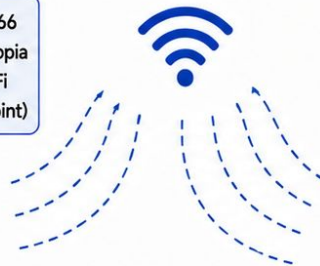


IP del servidor (ESP8266):
192.168.4.1

El ESP8266
crea su propia
red Wi-Fi
(Access Point)

Red Wi-Fi creada por
el ESP8266 (SoftAP)

STM32_WEB



Quiero
conectarme al
servidor



UART
Comandos AT
y datos



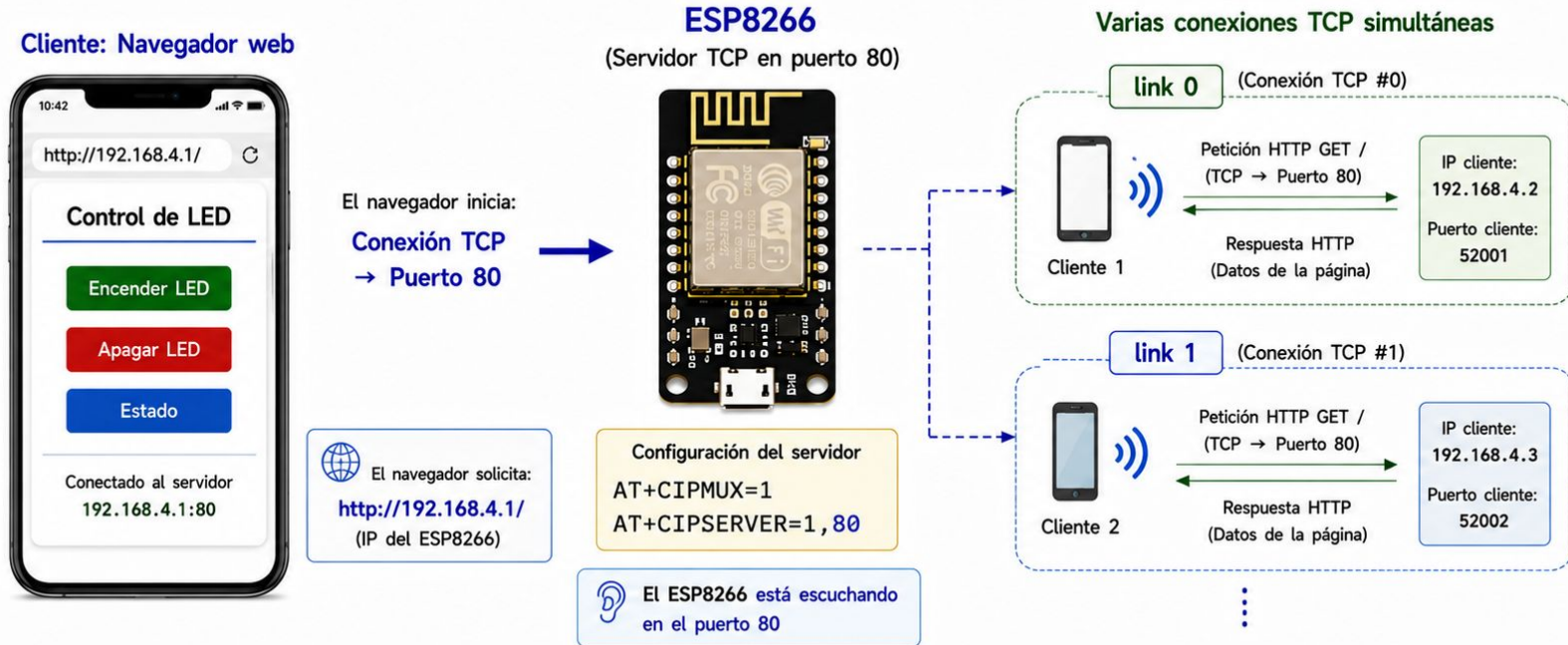
Wi-Fi (SoftAP)



La **IP** identifica al **dispositivo**
dentro de la red.

Servidor TCP en puerto 80

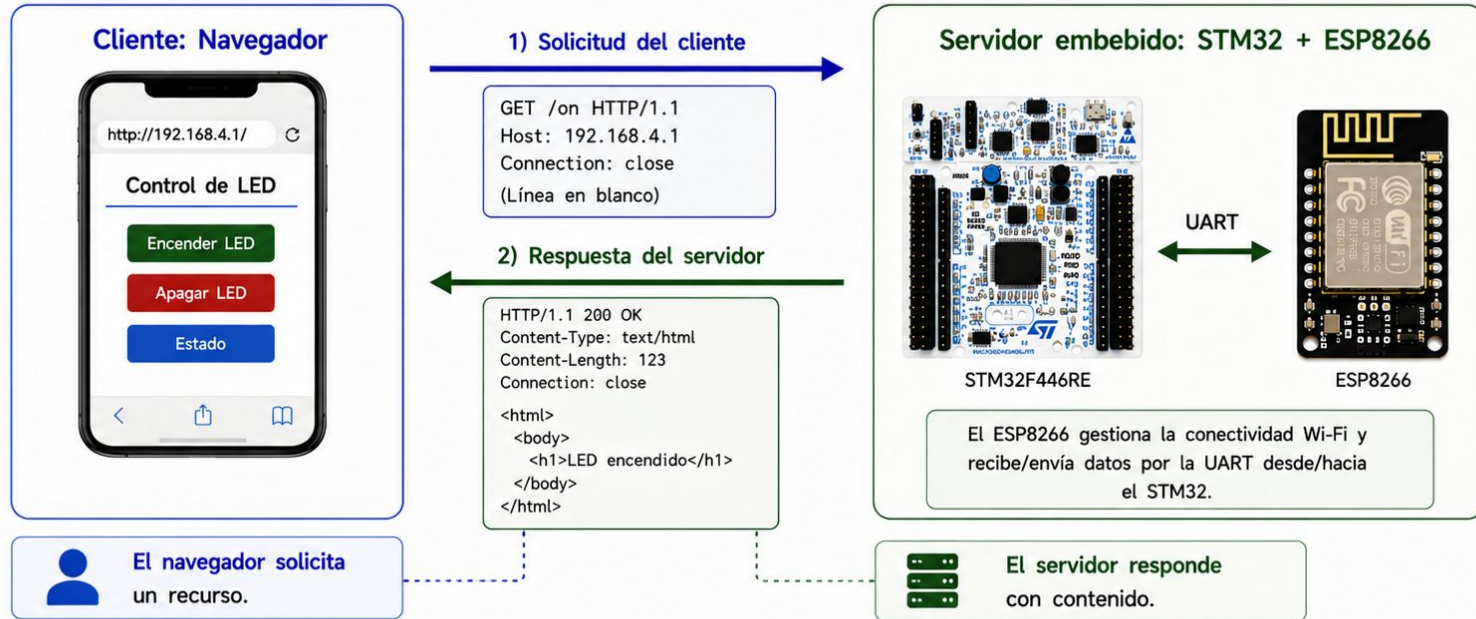
El ESP8266 escucha conexiones entrantes en el puerto 80 y atiende a varios clientes.



HTTP viaja sobre una conexión **TCP**.

HTTP: modelo solicitud-respuesta

HTTP es un protocolo de aplicación que permite al navegador solicitar recursos y al servidor responder con contenido.



HTTP es un protocolo de texto basado en **solicitud** y **respuesta**.

Rutas HTTP de la práctica

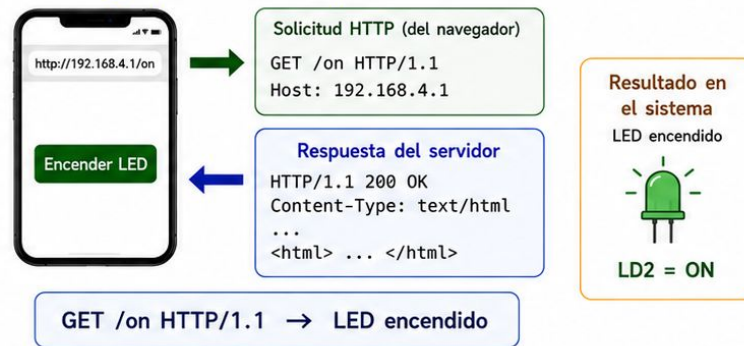
Estas rutas permiten controlar y consultar el estado de la salida LD2 desde el navegador.

RUTA	DESCRIPCIÓN	ACCIÓN EN EL STM32
 /	Página principal	 Devuelve la página HTML principal
 /on	Encender LD2	 Configura LD2 = ON (Salida en alto)
 /off	Apagar LD2	 Configura LD2 = OFF (Salida en bajo)
 /toggle	Invertir LD2	 Invierte el estado actual de LD2
 /status	Consultar estado	 Devuelve el estado actual (ON u OFF)

Estados del LED LD2



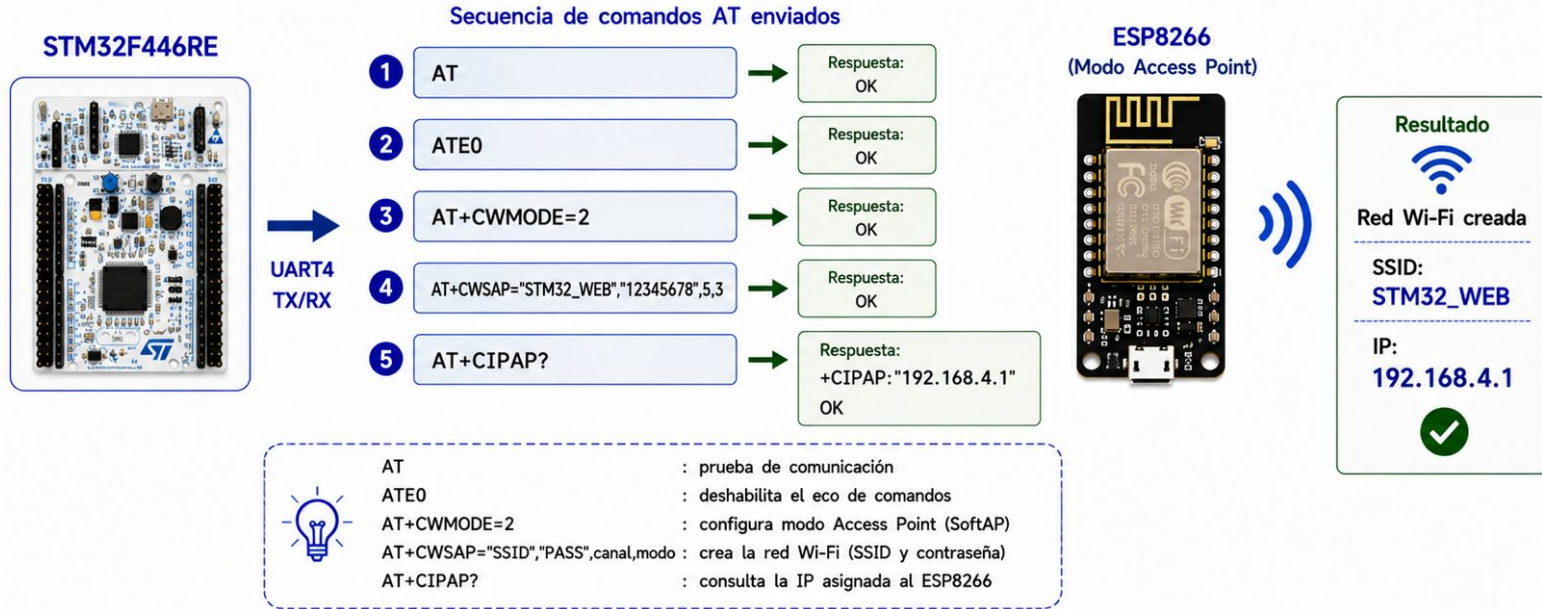
Ejemplo de solicitud y resultado



La **ruta solicitada** define la **acción** del **firmware**.

Configuración Wi-Fi con comandos AT

El STM32 envía comandos AT por UART4 para configurar el ESP8266 en modo Access Point (SoftAP).

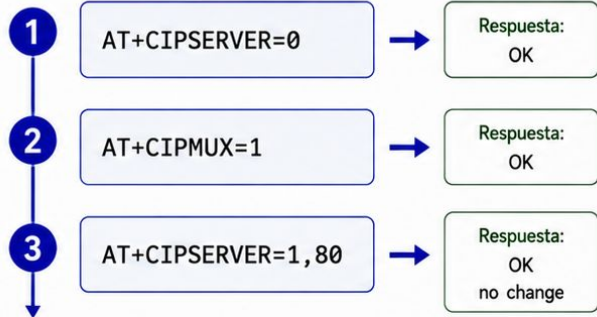


Los **comandos AT** permiten configurar el módulo **desde el microcontrolador**.

Habilitación del servidor TCP

El STM32 configura el ESP8266 para que acepte conexiones TCP entrantes en el puerto 80 (HTTP).

Secuencia de comandos enviados por el STM32 (UART4)



- 1 AT+CIPSERVER=0 : detiene cualquier servidor TCP activo.
- 2 AT+CIPMUX=1 : habilita múltiples conexiones y asigna identificadores (link ID).
- 3 AT+CIPSERVER=1,80 : inicia el servidor TCP en el puerto 80.

Estados del ESP8266

1. Servidor apagado
No acepta conexiones



CIPSERVER=0

2. Múltiples conexiones
Múltiples conexiones
habilitadas (CIPMUX=1)



CIPMUX=1

3. Servidor puerto 80 activo
Escucha conexiones TCP
entrantes en el puerto 80



CIPSERVER=1,80



Cientes web



Puerto 80 = HTTP
Protocolo web estándar.



CIPMUX=1
Permite múltiples conexiones simultáneas
identificadas por un "link ID".



El **ESP8266** escucha **conexiones web** en el **puerto 80**.

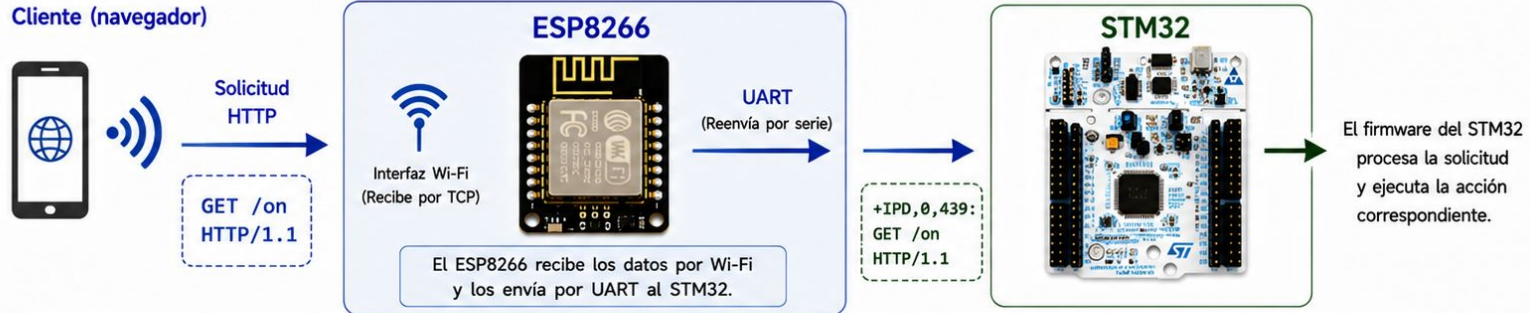
Cuando llega una solicitud: +IPD

El ESP8266 informa que recibió datos desde un cliente TCP (navegador).

Mensaje recibido por la UART desde el ESP8266



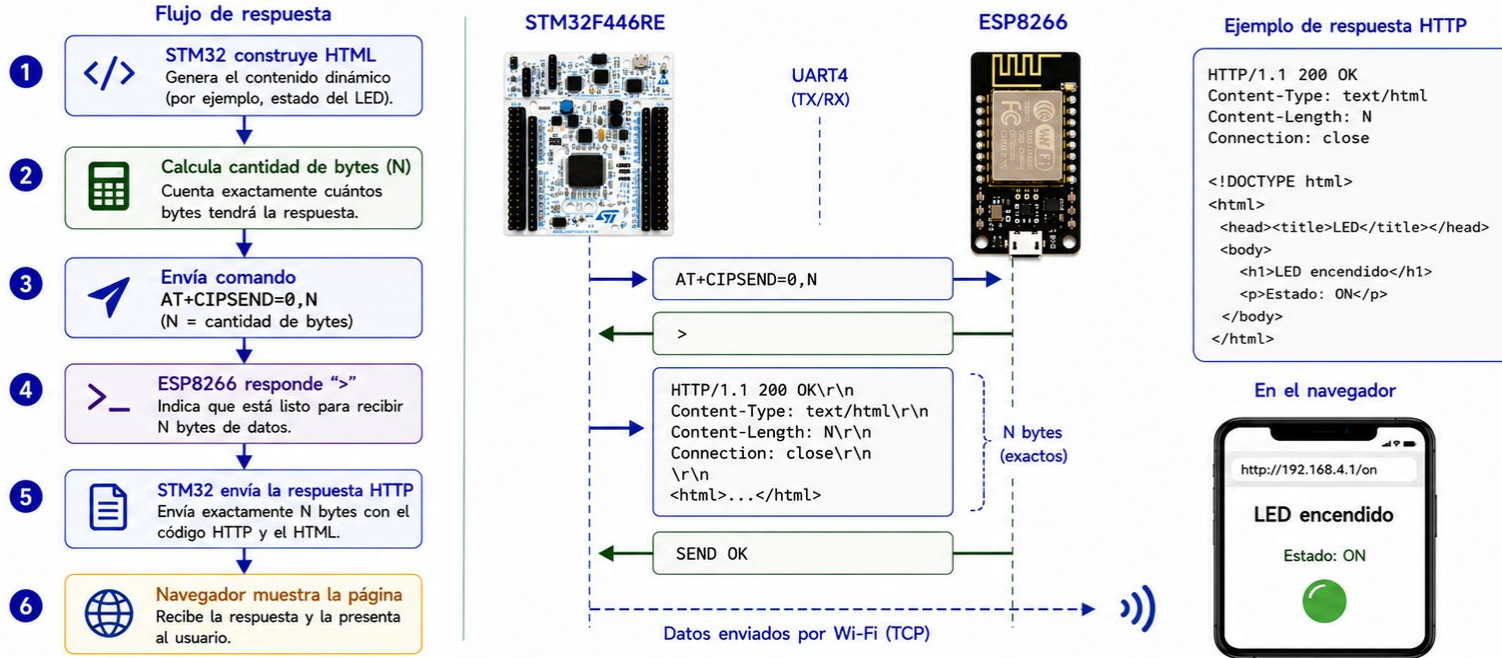
Cliente (navegador)



+IPD es el **aviso** del **ESP8266**: **llegaron datos desde la red.**

Respuesta HTTP desde el STM32

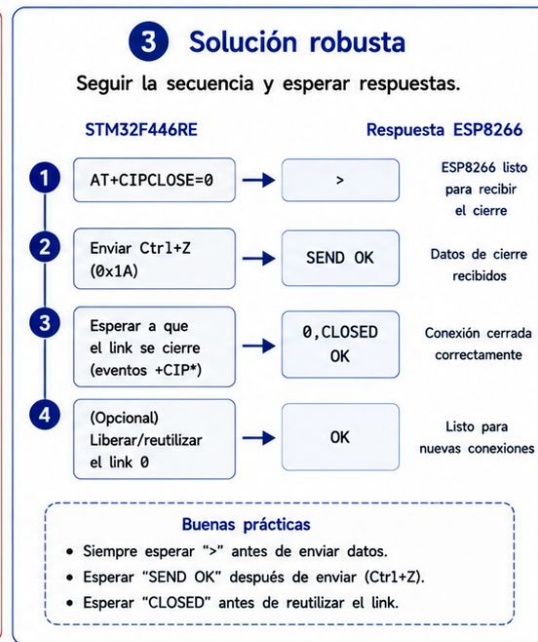
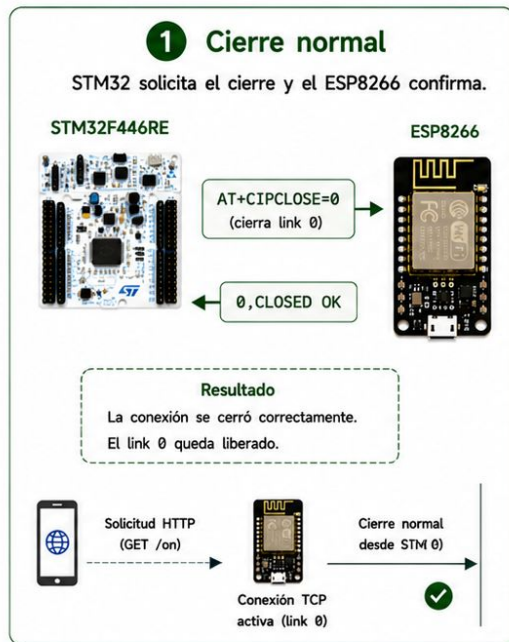
El STM32 genera la respuesta y la envía al navegador a través del ESP8266.



Antes de enviar datos, el **ESP8266** necesita saber **cuántos bytes** recibirá.

Cierre de conexión TCP

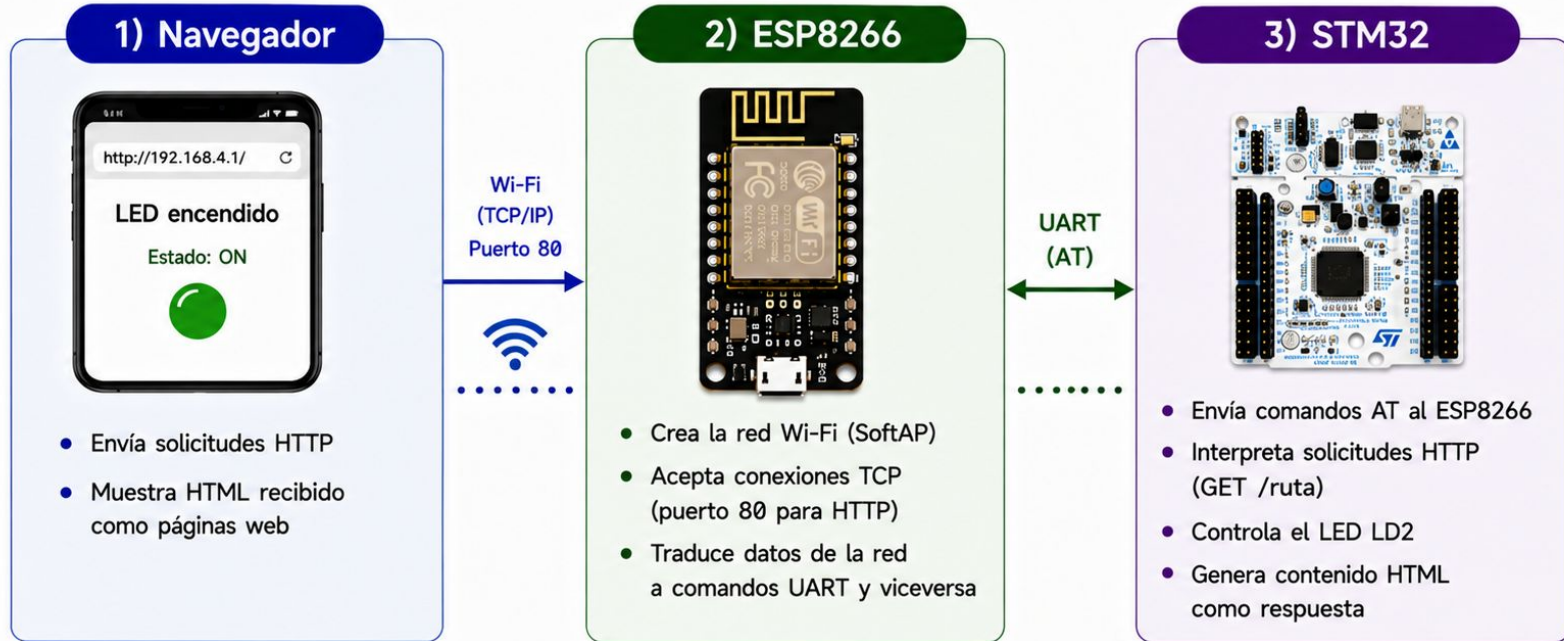
El STM32 cierra la conexión TCP enviando el comando AT+CIPCLOSE=<link> al ESP8266.



UNLINK indica que el **enlace TCP** ya se **cerró**.

Separación de responsabilidades

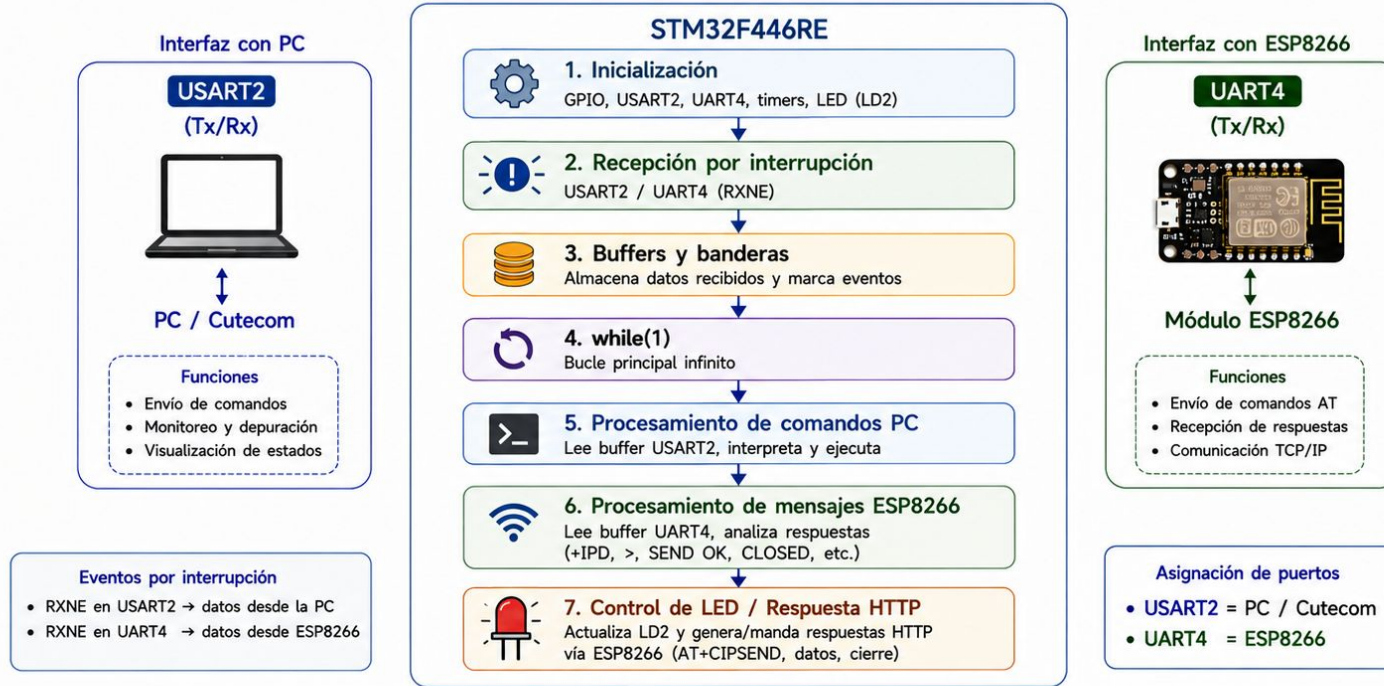
Cada parte del sistema tiene un rol específico en el servidor web embebido.



El **ESP8266** maneja la red; el **STM32** maneja la lógica de aplicación.

Estructura general del firmware

El firmware del STM32 organiza la comunicación con la PC y con el ESP8266 de forma concurrente.



La **interrupción** recibe; el lazo principal **procesa**.

Encuesta clase a clase